

Développer en Python avec GitHub Copilot

Formation GitHub Copilot pour le développement Python : des **completions inline** au **mode Agent**, via la personnalisation (**Instructions**, **Skills**).

MODULE	THÈME
1	Introduction
2	Completions inline
3	Chat et modes
4	Agent, Skills, Instructions + schéma
5	Path-specific, skills avancés, commit, review
6	Bonnes pratiques

Module 1 : Introduction à GitHub Copilot

Qu'est-ce que GitHub Copilot ?

GitHub Copilot est un assistant de programmation basé sur l'intelligence artificielle, développé par GitHub en collaboration avec OpenAI. Il fonctionne comme un **pair-programmer virtuel** intégré directement dans l'éditeur de code.

Principe de fonctionnement :

- Copilot analyse le contexte du code en cours d'écriture (fichiers ouverts, commentaires, noms de variables)
- Il génère des suggestions de code en temps réel, directement dans l'éditeur
- Le modèle sous-jacent a été entraîné sur des milliards de lignes de code provenant de dépôts publics GitHub
- Il est particulièrement efficace en Python grâce à l'écosystème massif open source (Django, FastAPI, Flask, data science, automatisation, etc.)

Ce que Copilot n'est pas :

- Ce n'est pas l'interpréteur Python ni mypy/ruff
- Il ne garantit pas que le code généré est correct ou sécurisé
- Il ne remplace pas la compréhension du langage Python et des bonnes pratiques PEP

Les différentes versions — du complétion à l'agent

GitHub Copilot propose plusieurs **niveaux d'autonomie**. La formation s'articule autour du mode **Agent** et de sa personnalisation (**Instructions, Skills**), tout en conservant les **completions inline** pour l'écriture au fil de l'eau.

VERSION	NIVEAU D'AUTONOMIE	DESCRIPTION	USAGE PRINCIPAL
Copilot (inline)	Faible	Suggestions de code directement dans l'éditeur	Complétion au quotidien, boilerplate
Copilot Chat	Moyen	Conversation (Ask, Edit, Plan)	Questions, explications, refactoring
Mode Agent	Élevé	Planifie, modifie plusieurs fichiers, exécute	Tâches multi-fichiers, debug, migration
Copilot CLI	Élevé	Agent en ligne de commande	Shell, tests, CI, scripts

Copilot inline reste le point d'entrée : dès qu'on tape du code, des suggestions apparaissent en gris (`Tab` pour accepter). Voir le **Module 2**.

Copilot Chat couvre Ask, Edit, Plan et Agent. Voir le **Module 3**.

Instructions, Skills et mode Agent : voir les **Modules 4 et 5**.

Copilot CLI (`gh copilot`) reprend la logique agentique hors de l'éditeur — utile pour lancer les tests (`pytest`), typer (`mypy`) ou formater (`ruff format`).

***Fil conducteur** : maîtriser les completions inline, puis le Chat, puis personnaliser le projet avec Instructions et Skills pour le mode Agent.*

Installation et configuration

Prérequis :

- Un compte GitHub avec un abonnement Copilot actif (Individual, Business ou Enterprise)

- Visual Studio Code installé
- Extension **Python** de Microsoft + **Pylance** (typage et IntelliSense)

Étapes d'installation :

1. Ouvrir VS Code
2. Aller dans Extensions (`Ctrl + Shift + X`)
3. Rechercher "GitHub Copilot" et installer l'extension
4. Installer également "GitHub Copilot Chat"
5. Se connecter à GitHub quand VS Code le demande
6. Vérifier l'icône Copilot dans la barre de statut (en bas)

Vérification du fonctionnement : Créer un fichier `test.py` et commencer à taper :

```
def greet(name: str) -> str:  
    """Affiche un message de bienvenue."""
```

Si Copilot fonctionne, une suggestion devrait apparaître en gris pour compléter la fonction.

Interface et statistiques d'utilisation

Pour consulter les statistiques d'utilisation de Copilot :

- Cliquer sur l'icône Copilot dans la barre de statut de VS Code
- Accéder au tableau de bord via GitHub : `Settings > Copilot > Usage`
- Les métriques disponibles : taux d'acceptation des suggestions, lignes de code générées, langages les plus utilisés

En entreprise (Copilot Business/Enterprise), les administrateurs ont accès à un dashboard détaillé montrant le pourcentage d'utilisation par équipe et par développeur.

Module 2 : Completions inline et contexte

Les **completions inline** sont le mode le plus utilisé au quotidien. Une fois les **Instructions** configurées (Module 4), elles produisent des suggestions alignées sur les standards du projet Python.

Raccourcis clavier essentiels

ACTION	RACCOURCI (WINDOWS/LINUX)	RACCOURCI (MAC)
Accepter la suggestion	Tab	Tab
Rejeter la suggestion	Échap	Échap
Suggestion suivante	Alt +]	Option +]
Suggestion précédente	Alt + [	Option + [
Accepter le mot suivant	Ctrl + →	Cmd + →
Déclencher manuellement	Alt + \	Option + \
Ouvrir le panneau de suggestions	Ctrl + Enter	Ctrl + Enter

Le panneau de suggestions (**Ctrl + Enter**) ouvre une fenêtre avec jusqu'à 10 suggestions alternatives. Utile quand la première suggestion ne convient pas.

Déclencher des suggestions

Commencer à taper une signature de fonction

```
def calculate_factorial(n: int) -> int:
```

Copilot va proposer le corps de la fonction en se basant sur le nom explicite et les types.

Écrire un commentaire descriptif

```
def bubble_sort(arr: list[int]) -> list[int]:  
    """Tri à bulles, retourne une nouvelle liste triée."""
```

Le commentaire guide Copilot sur l'algorithme attendu.

Créer une structure de données

```
@dataclass
class Employee:
    name: str
    age: int
    salary: float
```

Copilot pourra suggérer des méthodes cohérentes (`str`, `from_dict`, `to_dict`, etc.).

Nommer une variable de manière explicite

```
max_retry_count = 3
error_message: str | None = None
input_file = open("data.csv", encoding="utf-8")
```

Des noms de variables clairs aident Copilot à comprendre l'intention du code.

Le contexte compte

Copilot ne se base pas uniquement sur la ligne en cours. Il analyse un **contexte élargi** :

Les fichiers ouverts dans l'éditeur : Si vous avez `models.py` ouvert avec des dataclasses, Copilot les utilisera pour générer des services cohérents dans `services.py`.

Les imports influencent les suggestions :

```
from fastapi import APIRouter    # Copilot suggère des endpoints REST
import pandas as pd             # Copilot suggère du traitement de données
import asyncio                  # Copilot suggère du code async
```

Le code environnant guide la génération : Si les fonctions précédentes utilisent un style particulier (gestion d'erreurs, patterns async, immutabilité), Copilot va reproduire ce pattern.

```
# Si votre code existant fait ceci :
try:
    result = fetch_data(url)
except RequestError as exc:
    logger.error("Échec requête %s: %s", url, exc)
    raise

# Copilot reproduira ce pattern de gestion d'erreur
```

La fenêtre de contexte (context window)

La **fenêtre de contexte** (ou *context window*) est la quantité maximale de texte — code, commentaires, historique de chat, instructions du projet — que le modèle peut prendre en compte **en une seule requête**.

Définition concrète :

- Tout ce que Copilot « voit » avant de répondre occupe cette fenêtre : fichiers ouverts, sélection, messages du chat, `@workspace`, instructions Copilot, etc.
- Cette limite se mesure en **tokens** (morceaux de texte), pas en lignes de code.
- Au-delà de la limite, le contenu le plus ancien ou le moins prioritaire est **tronqué**.

Pourquoi c'est important :

CONSÉQUENCE	EXPLICATION
Perte de contexte	Un gros fichier + historique chat peuvent faire « oublier » le début.
Suggestions moins cohérentes	Si vos conventions ne tiennent plus dans la fenêtre, Copilot revient à des patterns génériques.
Réponses incomplètes en Agent	Sur un gros dépôt, l'agent doit cibler les bons fichiers.
Coût de qualité du prompt	Un contexte pertinent vaut mieux qu'un contexte volumineux.

Bonnes pratiques pour optimiser la fenêtre :

- Garder ouverts uniquement les fichiers **pertinents** (ex. `models.py` + `services.py`, pas tout le projet).
- Poser des questions **ciblées** dans le chat plutôt que de coller des milliers de lignes.
- Utiliser `#file:src/services/parser.py` pour un fichier précis plutôt que `#workspace` quand la question est locale.
- Découper les grosses refontes en étapes (module par module).
- Centraliser les règles du projet dans les **Instructions** (voir Module 4).

L'art du commentaire-prompt

En Python, les docstrings et les type hints guident Copilot.

Commentaire vague → résultat imprécis :

```
# trier le tableau
```

Commentaire précis → résultat ciblé :

```
def insertion_sort(arr: list[int]) -> list[int]:  
    """Tri par insertion, ordre croissant. O(n²) pire cas. Retourne une copie."""
```

Principes de base

Être spécifique et précis :

```
# ❌ Vague  
# lire un fichier  
  
# ✅ Précis  
def read_lines(file_path: str) -> list[str]:  
    """Lit un fichier texte ligne par ligne. Lève FileNotFoundError si absent."""
```

Donner du contexte :

```
def find_free_block(size: int) -> memoryview | None:  
    """Recherche un bloc libre dans la free list (stratégie first-fit)."""
```

Décomposer les problèmes complexes :

```
def parse_csv_line(line: str) -> list[str]: ...  
def token_to_employee(tokens: list[str]) -> Employee: ...  
def insert_employee(employees: list[Employee], emp: Employee) -> list[Employee]: ...
```

Itérer sur les suggestions : Si la première suggestion ne convient pas, utiliser `Alt + ]` pour voir les alternatives, ou reformuler le commentaire.

Structure d'un bon prompt

Un prompt efficace suit la structure **Quoi / Comment / Contraintes** :

```
def binary_search(arr: list[int], target: int) -> int:
    """
    QUOI : Recherche dans un tableau trié
    COMMENT : Dichotomie
    CONTRAINTES : arr trié croissant ; retourne l'index ou -1
    """
```

Autre exemple :

```
def deep_copy_list(head: Node | None) -> Node | None:
    """
    QUOI : Copie profonde d'une liste chaînée
    COMMENT : Parcours itératif, nouveaux nœuds
    CONTRAINTES : retourne None si head est None ; pas de mutation de l'original
    """
```

Itération et raffinement

Accepter partiellement une suggestion : Utiliser `Ctrl + →` (accepter mot par mot) quand le début de la suggestion est bon mais la suite diverge.

Modifier et relancer pour affiner :

```
# Premier essai – bubble sort basique
# Trier un tableau

# Deuxième essai – plus précis
def quicksort(arr: list[int]) -> list[int]:
    """Quicksort, pivot médian, fallback insertion si len < 10."""
```

Combiner plusieurs suggestions : Accepter une suggestion pour le squelette, puis supprimer certaines parties et redemander avec un commentaire plus spécifique.

Module 3 : Chat et modes d'interaction

Les modes du Chat — Ask, Edit, Plan, Agent

Copilot Chat propose plusieurs **modes** selon le niveau d'autonomie souhaité. Ils partagent les **Instructions** du projet ; seuls **Agent** et partiellement **Ask** exploitent les **Skills** (voir Module 4).

MODE	AUTONOMIE	COMPORTEMENT	EXEMPLE EN PYTHON
Ask	Faible	Répond, explique, ne modifie pas les fichiers	« Explique ce décorateur et son ordre d'application »
Edit	Moyenne	Modifie le code sélectionné ou le fichier actif	« Ajoute les type hints manquants sur cette fonction »
Plan	Moyenne	Produit un plan détaillé avant d'agir	« Plan pour migrer ce module vers async/await »
Agent	Élevée	Planifie, édite, exécute, itère	« Corrige toutes les erreurs mypy sur src/ »

Ask — comprendre du code sans modification :

- Sélectionner un bloc, poser une question : « Pourquoi j'ai une RecursionError ici ? »
- Les Instructions s'appliquent (ex. réponse alignée sur vos conventions docstring Google/NumPy)

Edit — changements localisés :

- Sélectionner une fonction, demander « /fix » ou une modification ciblée
- Plus rapide que l'Agent pour une modification ponctuelle

Plan — grosses tâches :

- Copilot produit un plan numéroté ; vous validez avant passage en Agent

Agent — cœur de l'approche agentique (voir Module 4) :

- Accès terminal, multi-fichiers, index sémantique
- Active automatiquement les **Skills** pertinents

Interface conversationnelle

Ouvrir le panneau Chat : `Ctrl + Shift + I` (ou `Cmd + Shift + I` sur Mac).

Exemples de questions utiles en Python :

- "Explique-moi ce générateur et son yield"
- "Pourquoi ce dataclass n'est pas hashable ?"
- "Comment implémenter un endpoint FastAPI avec validation Pydantic ?"
- "Génère les tests pytest pour cette fonction"

- "Optimise cette boucle pandas pour réduire les copies"

Obtenir des explications détaillées : Sélectionner un bloc de code complexe puis demander dans le chat : "Explique ce code étape par étape, en particulier la gestion des exceptions et les type hints"

Commandes slash

COMMANDE	ACTION
<code>/explain</code>	Explique le code sélectionné
<code>/fix</code>	Propose une correction pour le code sélectionné
<code>/tests</code>	Génère des tests pour le code sélectionné
<code>/doc</code>	Génère la documentation (docstrings Google ou NumPy)
<code>/new</code>	Crée un nouveau fichier/projet
<code>/clear</code>	Efface l'historique du chat

Exemple avec `/doc` :

```
def add_node(linked_list: LinkedList, data: object) -> int:
```

```
def add_node(linked_list: LinkedList, data: object) -> int:
    """
    Ajoute un nœud en tête de la liste chaînée.

    Args:
        linked_list: Liste cible.
        data: Données à stocker.

    Returns:
        Index du nœud créé, ou -1 en cas d'erreur.
    """
```

Sélection de contexte

Surligner un bloc de code, puis ouvrir le chat → Copilot comprend que la question porte sur ce code précis.

Indexation sémantique du codebase

L'**indexation sémantique** permet à Copilot de **comprendre le sens** du code du projet, pas seulement de faire correspondre des mots-clés.

Principe :

- Le dépôt est analysé en embeddings : fonctions, types, commentaires, relations entre fichiers.
- Une question du type « Où est gérée la validation ? » ou `@workspace trouve les handlers dupliqués` s'appuie sur cette index.
- Les résultats pertinents sont injectés dans la fenêtre de contexte.

APPROCHE	LIMITE
Fichiers ouverts + ligne courante	Ne couvre que ce que vous avez sous les yeux
Recherche par nom de symbole	Rate les implémentations sous un autre nom
Index sémantique	Retrouve du code par intention (« parsing CSV », « session SQLAlchemy », « gestion d'erreur HTTP »)

Bonnes pratiques :

- Laisser l'indexation se terminer après un clone ou un gros pull.
- Formuler des requêtes avec des **concepts** (« dataclass », « générateur », « context manager »).
- Combiner index sémantique + `#file:src/services/parser.py`.

Mode Agent en pratique

Le **mode Agent** est le point d'application des **Skills** et **Instructions** configurés au Module 4.

Copilot peut :

- Exécuter des commandes terminal (`pytest` , `mypy` , `ruff check` , `python -m`)
- Modifier plusieurs fichiers en séquence
- Itérer jusqu'à résoudre un problème ou signaler un blocage

Mode Agent : « **Corrige** toutes les erreurs mypy et ruff sur src/ »
→ **Copilot** active le skill **type-and-lint** (si présent)
→ **Applique** les **Instructions** (**PEP 8**, **type hints**)
→ **Lance** mypy et ruff, corrige les fichiers, relance pytest

Bonnes pratiques Agent :

- Formuler un **objectif mesurable** (« 0 erreur mypy et ruff sur src/ »)
- Laisser l'indexation sémantique se terminer sur les gros dépôts
- Vérifier manuellement le diff avant commit

Mode Cloud (aperçu)

Le mode Cloud permet d'exécuter des tâches Copilot sur l'infrastructure GitHub :

- Tâches longues en arrière-plan
- Pas besoin de garder VS Code ouvert
- Résultats via notification ou PR
- Utile pour des refactorings massifs ou des migrations

Module 4 : Architecture agentique — Agent, Skills, Instructions

Après les completions inline (Module 2) et le Chat (Module 3), ce module détaille la **personnalisation agentique** : Instructions, Skills et mode Agent.

Vue d'ensemble

CONCEPT	RÔLE	QUAND C'EST ACTIF	FICHER TIPIQUE
Instructions	Règles permanentes du projet	Toujours (inline, chat, agent)	<code>.github/copilot-instructions.md</code>
Skill	Workflow spécialisé, chargé à la demande	Quand la tâche correspond à la description	<code>.github/skills/<nom>/SKILL.md</code>
Agent	Mode autonome qui planifie et exécute	Sur demande explicite (mode Agent)	Interface Chat ou Copilot CLI

Les **Instructions** définissent *comment coder dans ce dépôt*. Les **Skills** enseignent *comment accomplir une tâche répétitive*. L'**Agent** orchestre le tout.

Qu'est-ce qu'un Agent ?

Un **agent** Copilot est un assistant **autonome** capable de :

- **Planifier** une tâche en plusieurs étapes
- **Lire et modifier** plusieurs fichiers du projet
- **Exécuter des commandes** dans le terminal
- **Itérer** jusqu'à un résultat satisfaisant

Contrairement à la **completion inline** ou au mode **Ask**, l'agent **agit** sur le dépôt.

Exemple concret en Python :

```
Mode Agent : « Corrigé toutes les erreurs mypy sur ce package »  
→ Lance mypy src/  
→ Analyse les diagnostics (fichier:ligne)  
→ Corrige les type hints et imports  
→ Relance pytest
```

Qu'est-ce qu'une Instruction ?

Les **Instructions** sont des consignes **permanentes** injectées à **chaque** interaction.

Elles répondent à : « *Quelles sont les règles de ce projet Python ?* »

FICHIER	PORTÉE
<code>.github/copilot-instructions.md</code>	Global — tout le dépôt
<code>.github/instructions/*.md</code>	Par chemin (applyTo: "src/**/*.py")
Instructions utilisateur (paramètres)	Tous vos projets

Exemple pour un projet Python :

Instructions pour ce projet Python

- Python 3.11+, type hints obligatoires sur l'API publique
- PEP 8 : *snake_case* pour fonctions/variables, *PascalCase* pour classes
- *Docstrings* Google sur les modules, classes et fonctions publiques
- Gestion d'erreurs : exceptions typées, pas de bare `except:`
- Préférer `pathlib` à `os.path` pour les chemins
- Exécuter mentalement ``mypy`` et ``ruff check`` avant de suggérer du code

Quand utiliser les Instructions :

- Conventions de nommage et style
- PEP 8, type hints, contraintes framework (Django ORM, Pydantic)
- Consignes pédagogiques (ne pas compléter les TODO des exercices)

Les Instructions restent **courtes et générales**. Pour un workflow détaillé, préférer un **Skill**.

Qu'est-ce qu'un Skill ?

Un **Skill** est un dossier avec `SKILL.md` et, optionnellement, scripts et références. Copilot le **charge** quand la tâche correspond à la description.

Structure :

```
.github/skills/  
├─ type-and-lint/  
│  └─ SKILL.md  
│  └─ scripts/  
│      └─ run_checks.sh  
├─ references/  
│  └─ patterns.md
```

Exemple de `SKILL.md` :

```

---
name: type-and-lint
description: Vérifie mypy et ruff sur un projet Python. Utiliser quand l'utilisateur mentionne mypy, ruff, type error, lint ou PEP 8.
---

## Workflow

1. Identifier la config (`pyproject.toml`, `mypy.ini`)
2. Exécuter `ruff check src/` puis `mypy src/`
3. Pour chaque erreur : localiser fichier:ligne, corriger types/style
4. Relancer `pytest`
5. Itérer jusqu'à 0 erreur mypy et ruff

## Patterns fréquents

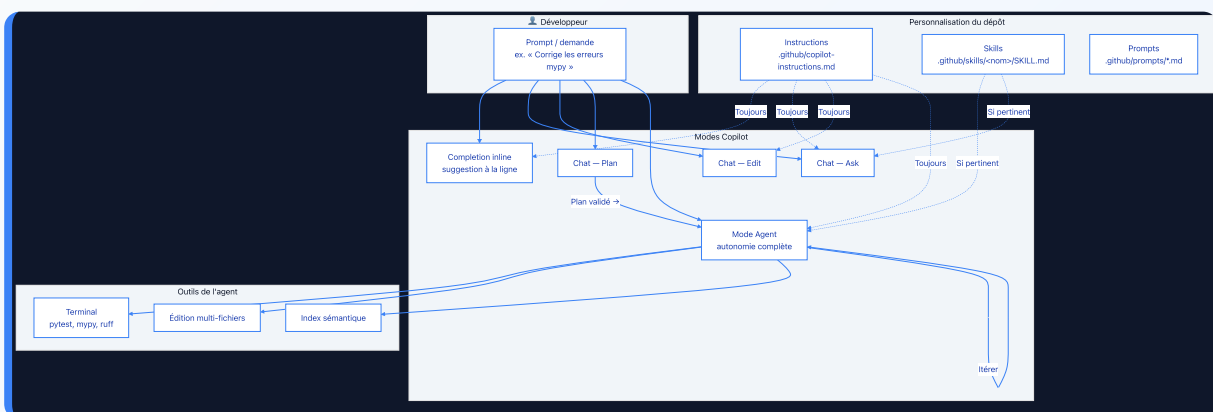
- Type hints manquants ou `Any` implicite
- Imports inutilisés ou ordre PEP 8 non respecté
- Exceptions trop larges (`except Exception`)

```

Instructions vs Skill :

	INSTRUCTIONS	SKILL
Contenu	Règles courtes, standards du projet	Workflow détaillé, scripts, références
Activation	Toujours	Seulement si la tâche est pertinente
Exemple	« type hints obligatoires, PEP 8 »	« Procédure complète mypy + ruff + pytest »

Schéma explicatif — comment tout s'articule



Lecture du schéma :

1. **Instructions** : toujours présentes — PEP 8, type hints, pas de bare except.
2. **Skills** : chargés à la demande.
3. **Agent** : terminal + éditions + itérations.
4. **Completion inline** : rapide, localisée ; bénéficie des Instructions.

Mise en place minimale pour un projet Python

Étape 1 — Instructions globales (`.github/copilot-instructions.md`)

Étape 2 — Instructions par chemin (`.github/instructions/tests.md` avec `applyTo:` `"tests/**/*.py"`) : Adapter aux tests (pytest, fixtures du projet).

Étape 3 — Un skill métier (`.github/skills/type-and-lint/SKILL.md`)

Étape 4 — Tester en mode Agent :

```
@workspace Corrige les erreurs mypy et ruff sur src/
```

Prompts réutilisables (complément)

```
<!-- .github/prompts/new-module.prompt.md -->
```

Crée un nouveau module Python avec :

- `__init__.py` exportant l'API publique`
- Types dans `types.py` ou annotations inline`
- Tests pytest dans `tests/test_<module>.py``
- Docstrings Google sur chaque export public

Module 5 : Path-specific, skills avancés, commit, review

Les concepts **Agent**, **Skill** et **Instruction** sont détaillés au **Module 4**.

Instructions spécifiques par chemin (path-specific)

Les instructions **path-specific** ne s'activent que lorsque Copilot travaille sur des fichiers correspondant à un **glob**.

Pourquoi les utiliser :

- L'API FastAPI (`src/api/`) n'a pas les mêmes règles qu'un script CLI (`scripts/`)
- Les tests (`tests/`) peuvent autoriser des fixtures/mocks interdits en production
- Les notebooks (`notebooks/`) vs code source (`src/`) demandent des consignes distinctes

Organisation recommandée :

```
---
applyTo: "src/**/*.py"
---

# Règles pour le code source Python

- Type hints sur les signatures publiques
- snake_case, docstrings Google
- Pas de bare except, préférer pathlib
```

```
---
applyTo: "tests/**/*.py"
---

# Règles pour les tests

- pytest uniquement, pas unittest sauf legacy
- Utiliser les fixtures de conftest.py
- Un fichier test_<module>.py par module testé
```

```
---
applyTo: "exercices/**/*.py"
---

# Contexte pédagogique – exercices étudiants

- Laisser les blocs TODO intacts
- Suggérer des indices en commentaire plutôt que des solutions complètes
- Respecter les noms de fonctions imposés par l'énoncé
```

Ordre de priorité :

1. Instructions globales
2. Instructions path-specific (glob correspondant)
3. Contexte immédiat (fichier, sélection, commentaire-prompt)

Bonnes pratiques :

- Préférer des globs **étroits** (`src/services/*.py`) à `**/*`
- Documenter *pourquoi* chaque règle existe
- Vérifier qu'une consigne globale n'annule pas une consigne locale

Créer des Skills avancés :

- Scripts dans `scripts/` (wrapper `ruff check` avec options du projet)
- Doc lourde dans `references/` pour préserver la fenêtre de contexte
- Affiner la `description` YAML — **déclencheur** de sélection

Prompts réutilisables (`.github/prompts/`)

```
<!-- .github/prompts/new-module.prompt.md -->
```

Crée un nouveau module Python avec :

- `__init__.py`` exportant l'API publique
- Types dans `types.py`` ou annotations inline
- Tests pytest dans `tests/test_<module>.py``
- Docstrings Google sur chaque export public

Commit automatique

- Cliquer sur l'icône Copilot dans Source Control
- Copilot analyse le diff et propose un message (Conventional Commits si configuré)

Exemple :

```
feat(parser): add CSV parsing with quoted field support
```

- Handle escaped quotes within fields
- Support multiline values enclosed in quotes
- Add error reporting with line numbers

Code review sur les commits en cours

- Source Control → "Review Changes" avec Copilot
- Particulièrement utile en Python pour détecter :
 - Injections SQL ou commandes shell via entrées utilisateur

- Usage de `eval`, `pickle` ou désérialisation non sûre
- Mutabilité par défaut (arguments list/dict mutables)
- Secrets ou tokens en dur dans le code

Fine tuning et personnalisation

- Les **Instructions** (Module 4) influencent toutes les suggestions
- Les **Skills** standardisent les workflows répétitifs
- Copilot apprend des patterns du code existant dans le dépôt

Module 6 : Bonnes pratiques et productivité

Validation du code généré

Le code Python généré par Copilot nécessite une vigilance particulière :

Toujours vérifier :

- Les type hints et cas None
- La gestion des exceptions (pas de bare except)
- Les effets de bord (mutabilité des listes/dicts par défaut)
- La concurrence (asyncio, GIL, race conditions)
- La sécurité (injection, secrets, désérialisation)

Outils de validation :

```
# Typage statique
mypy src/

# Lint et format
ruff check src/
ruff format --check src/

# Tests
pytest
```

Quand utiliser Copilot

Code répétitif ou boilerplate :

```
# Copilot excelle pour générer des fonctions CRUD similaires
def create_employee(data: CreateEmployeeDto) -> Employee: ...
def update_employee(emp_id: str, data: UpdateEmployeeDto) -> Employee: ...
def delete_employee(emp_id: str) -> None: ...
def get_employee_by_id(emp_id: str) -> Employee | None: ...
```

Algorithmes classiques, tests, exploration d'APIs : Copilot connaît les implémentations standards et le boilerplate (FastAPI, SQLAlchemy, pandas, asyncio).

Tests unitaires :

```
# Demander : "Génère les tests pytest pour binary_search"
def test_binary_search_found():
    assert binary_search([1, 3, 5, 7, 9], 7) == 3

def test_binary_search_not_found():
    assert binary_search([1, 3, 5, 7, 9], 4) == -1
```

Quand être prudent

Code critique pour la sécurité : toujours relire et tester en profondeur.

Logique métier complexe : Copilot peut produire du code syntaxiquement correct mais sémantiquement faux.

Contraintes spécifiques : embarqué, notebooks production, contraintes mémoire — Copilot ne connaît pas toujours le contexte d'exécution.

Performance critique : Copilot génère du code fonctionnel mais rarement optimal. Pour du NumPy/pandas intensif ou des boucles critiques, l'expertise humaine reste indispensable.

Productivité optimale

- Lire et comprendre chaque suggestion avant de l'accepter
- Copilot accélère l'écriture, il ne dispense pas de la réflexion
- Observer les suggestions pour découvrir des idiomes (découvrir `itertools`, `functools.lru_cache`, les dataclasses via les suggestions)

Adapter son workflow progressivement :

1. **Completions inline** — boilerplate (Module 2)
 2. **Chat Ask/Edit** — debugging et documentation (Module 3)
 3. **Instructions** — `.github/copilot-instructions.md` (Module 4)
 4. **Skills** — workflow récurrent (Module 5)
 5. **Mode Agent** — tâches multi-fichiers avec relecture du diff
-